

# Step 6: Git 打标签 + 推送

## 6.1 在终端依次执行以下命令

```
bash

git add index.html
git commit -m "perf: add lazy loading and optimize CSS fetch priority"
git tag -a v0.3-week2-performance -m "Week 2: performance optimization milestone"
git push
git push origin v0.3-week2-performance
```

## 6.2 逐行解读

| 命令                                                      | 含义                                                                                       |
|---------------------------------------------------------|------------------------------------------------------------------------------------------|
| <code>git add index.html</code> / <code>commit</code>   | 和往常一样，先把本日改动记录成一条提交                                                                      |
| <code>git tag -a v0.3-week2-performance -m "..."</code> | <code>-a</code> 表示创建一个"带注释的标签" (annotated tag)， <code>-m</code> 后面是标签说明；给当前最新的提交打上一个永久标记 |
| <code>git push</code>                                   | 照常推送本次的 commit                                                                           |
| <code>git push origin v0.3-week2-performance</code>     | 单独推送标签——标签默认不会随 <code>git push</code> 自动上传，需要专门推送                                        |

## 6.3 代码中完整的名称与关系列表

按代码出现顺序，逐个交代"它是什么身份、和谁发生什么关系":

| 出现顺序 | 名称                                                  | 它是什么身份                   | 和谁发生什么关系                                                 |
|------|-----------------------------------------------------|--------------------------|----------------------------------------------------------|
| 1    | <code>git tag</code>                                | 命令                       | 给某一个具体的 commit 打上一个永久的、易记的名字                             |
| 2    | <code>-a</code>                                     | <code>git tag</code> 的参数 | 决定创建的是"带注释标签"而不是最简单的"轻量标签", 带注释的标签会记录打标签的人、时间、说明文字       |
| 3    | <code>v0.3-week2-performance</code>                 | 你自定义的标签名                 | 从此刻起, 这个名字永久绑定到当前这条 commit 上, 即使以后又提交了新代码, 这个标签指向的位置也不会变 |
| 4    | <code>-m "..."</code>                               | <code>git tag</code> 的参数 | 给这个标签附加一段说明文字, 方便以后回顾"这个节点当时是什么状态"                       |
| 5    | <code>git push origin v0.3-week2-performance</code> | 单独针对标签的推送命令              | 把本地创建的这个标签, 同步到远程仓库; 如果不单独推送, 标签只存在于你自己电脑上               |

### 6.4 代码结构解析

text

git commit -m "..."

|

git tag -a v0.3-week2-performance -m "..."

|

└─ git push

|

└─ git push origin v0.3-week2-performance

← 【记录节点】生成一条新的本地提交 (假设 hash 是 abc123)

← 【打标节点】给 abc123 这个提交贴一个永久标签

← 【同步节点①】推送 commit 本身

← 【同步节点②】单独推送标签 (容易被遗漏的一步)

**关键节点说明：**打标签这个动作本身是本地操作，不会自动出现在远程仓库——这是新手最容易忽略的细节：以为 `git push` 会把标签也带过去，实际上普通的 `push` 只推送分支的提交历史，不包含标签，必须专门执行 `git push origin <标签名>`（或者用 `git push --tags` 一次性推送所有本地标签）。

6.5 标签 vs 普通提交对比

| 维度         | 普通 commit                         | Tag（标签）                                                                  |
|------------|-----------------------------------|--------------------------------------------------------------------------|
| 会不会随时间"前进" | 会，每次新提交都会产生新的 commit，分支指针跟着往前移    | 不会，标签一旦打上，永远指向那一个固定的历史节点                                                 |
| 典型用途       | 记录日常每一次改动                         | 标记"里程碑"，比如版本发布、阶段性完成                                                     |
| 是否需要单独推送   | 不需要（ <code>git push</code> 会自动带上） | 需要（必须单独 <code>git push origin &lt;标签名&gt;</code> 或 <code>--tags</code> ） |

6.6 生活化类比

commit 就像日记里每天的流水账，一页页往后翻；`tag` 则像在日记本里的某一页贴上一张书签，写着"这是我完成 Week 2 学习的那一天"——无论日记后面又写了多少页，这张书签始终指向那一页不会挪动，方便以后快速翻回去查看那个"当时的状态"。

6.7 完成标准

- 1. 依次执行上面 5 条命令
- 2. 在 GitHub 仓库页面，点进 "Tags"（通常在 commits 旁边或仓库主页的分支下拉菜单里）

你应该看到：`v0.3-week2-performance` 这个标签出现在远程仓库的标签列表里。

完成后请回复：

- "推送成功，标签能看到，进入下一步"
- 或 "遇到问题：\_\_\_\_"